

SkillForge DSA (Data Structures and Algorithms) Mastery

Most Common & Important DSA Interview Questions with Answers - 50 MCQs with Explanations

Set 1: Beginner + Intermediate — 30 Questions

This set covers fundamentals: arrays, linked lists, stack, queue, hashing, trees, searching, sorting, recursion and basic complexity.

1. What is a Data Structure?

- A) A way to organize, store and manage data efficiently
- B) A programming language
- C) A hardware component
- D) Only an algorithm

Correct Answer: A - Data structure is a format for organizing, processing, retrieving and storing data to enable efficient access and modification.

2. What is the key difference between Array and Linked List?

- A) Array is fixed-size contiguous memory, Linked List is dynamic non-contiguous with pointers
- B) Both are same
- C) Array is dynamic, Linked List is fixed
- D) Linked List uses contiguous memory

Correct Answer: A - Array provides $O(1)$ random access but fixed size. Linked List allows $O(1)$ insertion/deletion at head with dynamic size but $O(n)$ access.

3. Which data structure follows LIFO (Last-In-First-Out) principle?

- A) Queue
- B) Stack
- C) Tree
- D) Graph

Correct Answer: B - Stack - push and pop from same end. Used for recursion, undo, expression evaluation. Queue is FIFO.

4. Which data structure follows FIFO (First-In-First-Out) principle?

- A) Stack
- B) Queue
- C) Heap
- D) Trie

Correct Answer: B - Queue processes in arrival order. Used for BFS, scheduling, buffering.

5. What is the time complexity of Binary Search on a sorted array of n elements?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(n \log n)$
- D) $O(1)$

Correct Answer: B - Binary Search halves search space each iteration, so $\log_2 n$ comparisons.

6. What is the worst-case time complexity of QuickSort?

- A) $O(n \log n)$
- B) $O(n^2)$
- C) $O(n)$
- D) $O(\log n)$

Correct Answer: B - When pivot is smallest/largest every time (sorted array with first/last pivot), partition is unbalanced leading to $O(n^2)$. Average is $O(n \log n)$.

7. Which sorting algorithm is stable?

- A) QuickSort
- B) HeapSort
- C) Merge Sort
- D) Selection Sort

Correct Answer: C - Stable preserves order of equal keys. Merge Sort, Insertion, Bubble are stable. QuickSort, HeapSort, Selection are unstable by default.

8. Which data structure is used internally for recursion?

- A) Queue
- B) Stack (call stack)
- C) Heap
- D) Graph

Correct Answer: B - System maintains call stack of activation records for function calls, enabling recursion.

9. What is the time complexity of inserting at the head of a singly linked list?

- A) $O(n)$
- B) $O(1)$
- C) $O(\log n)$
- D) $O(n^2)$

Correct Answer: B - Only head pointer update: `newNode->next = head; head = newNode.`

10. What does load factor mean in Hashing?

- A) n/m where n =elements, m =table size
- B) m/n
- C) Number of collisions
- D) Hash function size

Correct Answer: A - Load factor $\alpha = n/m$. High α means more collisions. Rehash when $\alpha > \text{threshold}$ (e.g., 0.75).

11. Which collision resolution technique uses linked lists at each bucket?

- A) Linear Probing
- B) Separate Chaining
- C) Double Hashing
- D) Quadratic Probing

Correct Answer: B - Separate Chaining stores colliding keys in linked list/tree at bucket. Probing methods use empty slots.

12. In-order traversal of a BST yields?

- A) Nodes in ascending sorted order
- B) Descending order
- C) Random order
- D) Insertion order

Correct Answer: A - In-order is Left-Root-Right. For BST, it visits keys in sorted order.

13. What is the height of a balanced binary tree with n nodes?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) $O(n^2)$

Correct Answer: B - Balanced tree maintains height $\sim \log_2 n$, ensuring $O(\log n)$ operations.

14. Which data structure is used for BFS traversal of a graph?

- A) Stack
- B) Queue
- C) Heap
- D) Array

Correct Answer: B - BFS explores level by level using Queue. DFS uses Stack/recursion.

15. Which data structure is used for DFS traversal?

- A) Queue
- B) Stack / Recursion
- C) Heap
- D) Linked List

Correct Answer: B - DFS goes deep first using Stack or recursion.

16. What is the time complexity of accessing an element by index in an array?

- A) $O(n)$
- B) $O(1)$
- C) $O(\log n)$
- D) $O(n \log n)$

Correct Answer: B - Array base address + index*size gives direct access.

17. Which algorithm is best to detect a cycle in a linked list?

- A) Floyd's Tortoise and Hare (two pointers)
- B) Binary Search
- C) BFS
- D) Linear Search

Correct Answer: A - Slow moves 1 step, fast moves 2 steps. If they meet, cycle exists. $O(n)$ time, $O(1)$ space.

18. What is the space complexity of Merge Sort?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Correct Answer: B - Needs auxiliary array of size n for merging.

19. Which data structure is used to evaluate postfix expressions?

- A) Queue
- B) Stack
- C) Tree
- D) Graph

Correct Answer: B - Scan left to right, push operands, pop two when operator found, push result.

20. What is the advantage of doubly linked list over singly?

- A) Bidirectional traversal and $O(1)$ deletion if pointer given
- B) Less memory
- C) Faster random access
- D) No advantage

Correct Answer: A - Doubly has prev and next, allows forward/backward traversal and quick deletion.

21. What is worst-case time complexity of Bubble Sort?

- A) $O(n \log n)$
- B) $O(n^2)$
- C) $O(n)$
- D) $O(\log n)$

Correct Answer: B - Nested loops: $n*(n-1)/2$ comparisons.

22. Which searching algorithm works on an unsorted array?

- A) Binary Search
- B) Linear Search
- C) Interpolation Search
- D) Jump Search

Correct Answer: B - Linear Search $O(n)$ scans each element, doesn't require sorting.

23. If you push 1,2,3 onto a stack and pop all, what is output order?

- A) 1,2,3
- B) 3,2,1
- C) 2,1,3
- D) 2,3,1

Correct Answer: B - Stack LIFO reverses order.

24. What is an ADT (Abstract Data Type)?

- A) Type defined by behavior/operations, not implementation
- B) Array Data Type
- C) Algorithm Data Type
- D) Application Data Type

Correct Answer: A - ADT defines what operations do (e.g., Stack push/pop), implementation can be array or linked list.

25. What does Big O notation represent?

- A) Upper bound on worst-case growth rate
- B) Exact running time
- C) Only memory usage
- D) Best case only

Correct Answer: A - Big O describes asymptotic upper bound as n grows.

26. Which combination is used for basic LRU Cache implementation?

- A) Array + Stack
- B) HashMap + Doubly Linked List
- C) Queue + Tree
- D) Heap + Array

Correct Answer: B - HashMap for O(1) lookup, DLL for maintaining recency and O(1) removal.

27. What is advantage of Circular Queue over linear queue?

- A) Efficient memory reuse, avoids shifting
- B) Slower
- C) Uses more memory
- D) No advantage

Correct Answer: A - Circular queue wraps rear to front, reusing empty slots.

28. Balanced parentheses problem is solved using?

- A) Stack
- B) Queue
- C) Heap
- D) Tree

Correct Answer: A - Push opening brackets, pop when closing matches.

29. Main difference between BFS and DFS?

- A) BFS level-wise using Queue, DFS depth-wise using Stack
- B) Both same
- C) BFS uses Stack
- D) DFS uses Queue

Correct Answer: A - BFS finds shortest path in unweighted graph; DFS useful for topological sort, cycle detection.

30. What is time complexity of Linear Search?

- A) $O(\log n)$
- B) $O(n)$
- C) $O(1)$
- D) $O(n^2)$

Correct Answer: B - May need to scan all n elements in worst case.

Set 2: Professional / 10+ Years Experience — 20 Questions

This section focuses on production-grade trade-offs: storage engines, concurrent data structures, Bloom filters, skip lists, persistent structures, pagination, caching and senior design judgment.

1. For high write throughput with range scans at scale, which storage engine is preferred?

- A) B+ Tree - read optimized
- B) LSM Tree - write optimized with memtable + SSTables + compaction
- C) Hash Table only - no range
- D) Simple Array

Correct Answer: B - LSM buffers writes in memory and flushes to SSTables, compaction amortizes. Used in RocksDB, Cassandra. B+ Tree has write amplification due to in-place updates.

2. How to achieve $O(1)$ LFU (Least Frequently Used) Cache?

- A) HashMap + DLL only
- B) HashMap freq $\rightarrow$ DLL list + HashMap key $\rightarrow$ node + minFreq tracker
- C) Array only
- D) Heap only $O(\log n)$

Correct Answer: B - Maintain map of frequency to DLL of nodes with that freq, plus minFreq to evict. Both get and put $O(1)$.

3. What is expected search time in Skip List?

- A) $O(n)$
- B) $O(\log n)$ expected
- C) $O(1)$
- D) $O(n \log n)$

Correct Answer: B - Probabilistic multi-level linked list with 50% promotion. Expected $O(\log n)$ search/insert/delete, simpler than balanced trees.

4. What is consistent hashing mainly used for?

- A) Sorting numbers
- B) Distributed caching / load balancing with minimal rehashing when nodes added/removed
- C) Encryption
- D) Graph traversal

Correct Answer: B - Hash ring with virtual nodes minimizes key movement when cluster scales. Used in Dynamo, Cassandra, CDN, Memcached.

5. Best way to detect cycle in large directed graph with millions of nodes in production?

- A) DFS with 3-color marking (white/gray/black) + iterative stack to avoid recursion overflow
- B) BFS only
- C) Linear scan
- D) Bubble sort

Correct Answer: A - Gray node visited in current path -> back edge = cycle. Iterative avoids stack overflow; Kahn's algorithm also works.

6. Bloom Filter properties?

- A) No false positives, may have false negatives
- B) May have false positives, never false negatives
- C) No errors at all
- D) Both false positives and negatives

Correct Answer: B - Bloom is space-efficient probabilistic structure. If it says NOT present, definitely not present. If says present, may be false positive.

7. How to implement a concurrent HashMap for high throughput?

- A) Single global lock
- B) Segmented locking / lock striping + CAS on buckets
- C) No locking
- D) Use array only

Correct Answer: B - Modern CHM uses 16+ segments or synchronized buckets + volatile reads and CAS for lock-free reads, allowing parallel writes.

8. Best data structure for autocomplete at scale (100M queries)?

- A) HashMap
- B) Trie with top-k min-heap at each node + caching + compression
- C) Linked List
- D) Stack

Correct Answer: B - Trie gives prefix search $O(m)$. Each node stores popular suggestions in heap. Use DAWG/radix optimization and Redis cache.

9. How to handle hash flooding / collision DoS attack?

- A) Randomized hash seed + convert bucket to balanced BST when threshold exceeded
- B) Use only arrays
- C) Ignore
- D) Disable hashing

Correct Answer: A - Java 8 HashMap uses random seed and treeifies bucket to $O(\log n)$ after 8 collisions. Also use SipHash.

10. What is a persistent segment tree?

- A) Tree that keeps all historical versions after updates via path copying
- B) Stored only on disk
- C) Never updates
- D) Normal segment tree

Correct Answer: A - Path copying creates $O(\log n)$ new nodes per update, old root preserved. Used for k-th order statistics over versions.

11. For sparse graph with 10M nodes and 20M edges, which representation?

- A) Adjacency Matrix $O(V^2) = 100T$ entries
- B) Adjacency List $O(V+E)$ memory efficient
- C) Edge matrix only
- D) Complete matrix

Correct Answer: B - List stores only existing edges. Matrix impossible for large sparse graph.

12. What data structure is used for token bucket rate limiter?

- A) Counter + timestamp / Redis Sorted Set for sliding window log
- B) Stack only
- C) Tree only
- D) Array only

Correct Answer: A - Token bucket uses counter refilled at fixed rate. Sliding window uses sorted set with timestamps. Implemented with Redis INCR + TTL.

13. What is a cache-oblivious algorithm?

- A) Ignores cache
- B) Optimized for any cache size without knowing B and M parameters, uses divide-and-conquer
- C) Only for L1 cache
- D) Disables cache

Correct Answer: B - Performs well at all memory hierarchy levels without tuning. Example: cache-oblivious matrix multiplication.

14. Why is OFFSET pagination bad at scale and what is better?

- A) OFFSET is fast
- B) OFFSET scans and discards rows $O(n)$; better is keyset pagination `WHERE id > last_id LIMIT N`
- C) OFFSET uses no memory
- D) OFFSET is $O(1)$

Correct Answer: B - Deep OFFSET is expensive. Keyset pagination uses indexed seek. For random access use deferred join.

15. What differentiates a 10-year DSA engineer decision?

- A) Only knows syntax
- B) Evaluates trade-offs: time vs space vs cache locality vs concurrency vs maintainability
- C) Always uses $O(n^2)$
- D) Prefers brute force

Correct Answer: B - Senior considers hardware, data size, concurrency, readability, and business constraints, not just Big-O.

16. When to use Trie vs HashMap for dictionary with prefix queries?

- A) HashMap always better
- B) Trie for prefix/autocomplete/spell-check, HashMap for exact $O(1)$ lookup
- C) Trie never useful
- D) Both same

Correct Answer: B - Trie excels for prefix search and lexicographic order but higher memory. HashMap better for exact key lookup.

17. What is time complexity of Union-Find with path compression and union by rank?

- A) $O(\log n)$
- B) $O(\alpha(n))$ inverse Ackermann - almost $O(1)$ amortized
- C) $O(n)$
- D) $O(n \log n)$

Correct Answer: B - $\alpha(n) < 5$ for practical n , so nearly constant. Used for Kruskal, dynamic connectivity.

18. Why does Dijkstra's algorithm fail with negative edge weights?

- A) Greedy assumes finalized distance never improves, negative edges violate this
- B) It works with negatives
- C) Fails with positive weights
- D) Fails with cycles only

Correct Answer: A - Dijkstra marks node visited as final shortest. Negative edge can later give shorter path, breaking invariant. Use Bellman-Ford.

19. AVL vs Red-Black tree trade-off?

- A) AVL more strictly balanced, faster lookup; Red-Black fewer rotations, faster insertion/deletion
- B) Red-Black more balanced
- C) Both identical
- D) AVL faster insertion

Correct Answer: A - AVL height $< 1.44 \log n$ vs Red-Black $< 2 \log n$. So AVL lookup faster, but more rotations on updates.

20. In database buffer pool, LRU vs LFU vs ARC trade-off?

- A) LRU good for recency, LFU for frequency, ARC adaptively balances both - best for DB buffer pool
- B) LRU always best
- C) LFU always best
- D) No difference

Correct Answer: A - Real workloads have both recency and frequency. ARC (Adaptive Replacement Cache) tracks both and adapts, used in PostgreSQL, MySQL.